Security Documentation

Etel Delivery & Ride Platform

Prepared for INSA Security Audit Submission (via Hibir Platform)

Etel-FTBG · 2026-09-22

INSA Pre-Submission Security Findings — Etel Delivery & Ride

Prepared to fill the `[TO COMPLETE]` placeholders in `INSA_Audit_Package_Etel_Delivery_Ride.docx` (Sections 5.1–5.4, 6, 7, 8.3). Scope: `delivery/` (the Etel Delivery & Ride PHP/MySQL backend + its front-end apps), living in this repo at `https://github.com/Etel-FTBG/Etel-FTBG`. No deployment was performed — everything below is a code-level fix in the git working tree. Deployment to `ethelmarket.com/delivery/` happens separately, through the Admin Updater tool (`delivery/admin.html` → Updater tab), uploaded manually by Dagi.

Every finding below was produced by reading the actual code (file/function names cited throughout), not inferred. Every `.php` file edited was verified with `php -l` (no syntax errors) and checked for non-ASCII bytes introduced by the edit (`python3 -c "print([ch for ch in open(f).read() if ord(ch)>127])"` → must be `[]`) per this host's WAF constraint. `.js` / `.html` files are not subject to the ASCII constraint.

---

1. Authorization enforcement (Section 5.1)

Checked: every action in `OrderController.php`, `RideController.php`, `AuthController.php`, `ServiceGiverController.php`, `AgentController.php` — full action list in the per-file report — for whether the caller's role and, where relevant, the caller's ownership of the specific row being read/written is verified server-side.

Fixed

Two IDOR (Insecure Direct Object Reference) findings in `OrderController.php`, both **Low severity**:

- **`check_pickup_confirmation`** and **`check_delivery_confirmation`** required only `require_courier($pdo)` — any authenticated courier — with no check that the `order_id` passed in the request actually belonged to *that* courier. Any courier could pass an arbitrary `order_id` and read another courier's pickup/delivery confirmation status (and, for the delivery case, the `payment_confirmed` flag). Low severity because the only data exposed is a status string and a timestamp, not PII or money — but a real cross-account read with zero ownership check.
- **Fix applied:** added the same ownership check the sibling actions (`request_pickup_confirmation` / `request_delivery_confirmation`) already used one case above each: `php $cu = require_courier($pdo); $courier = get_or_create_courier_profile($pdo, $cu['id']); if (!$courier) fail('Courier profile not found', 404); // ... (order_id parsing, unchanged) ... $ownChk = $pdo->prepare('SELECT id FROM orders WHERE id=? AND courier_id=?'); $ownChk->execute(array($order_id, $courier['id'])); if (!$ownChk->fetch()) fail('Order not found or not assigned to you', 403);` File: `delivery/api/controllers/OrderController.php`. Verified with `php -l` and the ASCII check (only pre-existing box-drawing `—` characters in section-header comments remain, none introduced by this edit).

Verified clean

- **`AuthController.php`** **`register`**: client-supplied `role` is hard-restricted — `if (!in_array($role, array('customer', 'courier'))) $role = 'customer';` — no path to self-register as `admin` or `service_giver`.
- **`AuthController.php`** **`admin_etelid_login`**: only promotes to `role='admin'` after a server-side check (existing admin row, or a live role lookup against EtelGroup's own `etelid_bridge_check_role('etel-delivery', ...)`) — never trusts the request body for this decision.
- **`AuthController.php`** **`change_password`**: requires auth, verifies `current_password` via `password_verify()`, bumps `token_version` to revoke old tokens.
- **`OrderController.php`**: every courier-only action resolves the caller's own courier profile via `get_or_create_courier_profile()` and scopes queries to that courier's own `courier_id`; every customer action scopes to `customer_id = $auth_user['id']` (or an explicit ownership `SELECT` before any write); every admin action checks `role === 'admin'` (`admin_financial_report` additionally requires the `finance.view` capability).
- **`ServiceGiverController.php`**: every action funnels through `sg_get_context()` + `sg_require_capability()` / `sg_require_owner()`, and every order/item/staff/role mutation additionally verifies the target row's `service_giver_id` matches the caller's own business before acting. The one place a platform admin bypasses the owner-only check is explicitly and intentionally gated behind `role === 'admin'` plus an admin-supplied `sg_id` (documented inline as deliberate).
- **`RideController.php`**: every ride-scoped action verifies `driver_id` / `passenger_id` ownership (or, for shared actions like `send_ride_message`, computes the caller's role from whichever column matches and fails 403 if neither matches and the caller isn't admin) before reading or mutating; all `admin_*` actions in this file check `role === 'admin'` plus the relevant capability.
- **`AgentController.php`**: every action calls `agent_verify_service_key()` first (constant-time `hash_equals()` against `ETELID_SERVICE_KEY`) — a customer bearer token alone cannot reach any `agent_*` action. Member-specific actions additionally require a valid `etel_sso_token` resolved to a specific local user, and every subsequent query is scoped to that resolved user's own id. No path lets the service key alone act on an arbitrary member's data, and no path lets a plain customer token perform agent-privileged actions.

Needs Dagi's judgment call

`RideController.php` : driver-privileged actions are reachable by any authenticated role, including a plain `customer` account, with no identity-verification gate before accepting a real, paying ride.

Unlike `OrderController.php` (`require_courier()`) or `ServiceGiverController.php` (`sg_get_context()`), `RideController.php` has no `require_driver()` -equivalent anywhere — the only gate is the file-level `require_auth($pdo)` (any logged-in user, any role). This means a plain `customer` -role account can call `driver_go_online` , `driver_get_nearby_requests` , `driver_accept_ride` , `driver_start_ride` , `driver_arrived` , `driver_complete_ride` , `driver_cashout` , `driver_topup_etel_money` , `driver_hail_customer` , etc. — accepting and running a real passenger's ride end-to-end and collecting real money — with no check that `ride_drivers.onboarded = 1` (vehicle info, selfie, license photo, EtelID identity cross-check all confirmed).

The code has extensive comments (around `RideController.php` lines 2597–2608) documenting this as a deliberate progressive-onboarding design: `users.role` is only promoted to `'driver'` *after* onboarding completes, and nothing currently blocks the driver-action endpoints from running before that promotion. This was **not auto-fixed** because (1) it's explicitly documented as intentional, not an oversight; (2) there is no existing pattern in this file to mechanically extend — building a real gate here means new business logic, not a copy of an established pattern; (3) the fraud/safety stakes are real enough (an unverified account driving real passengers and handling real money) that exactly where the line should be drawn — `driver_go_online` itself, or only the money-moving actions, or widening the existing `identity_match==='mismatch'` block — is a product decision with real onboarding-UX tradeoffs, not something to guess at.

---

2. SQL injection review (Section 6)

Checked: every `$pdo->prepare()` / `$pdo->query()` / `$pdo->exec()` call in `OrderController.php` (118), `RideController.php` (183), `AuthController.php` (19), `ServiceGiverController.php` (72), `AgentController.php` (13), `AdminController.php` (147) — traced each back from request input through to where it lands in the SQL string. Also spot-checked `boot.php` 's `admin_get_users` / `admin_get_orders` / `admin_get_couriers` / `admin_update_courier` (technically outside the assigned file list, but exactly the kind of admin search/filter endpoint this section is concerned with, and referenced by `AdminController.php` 's own comments as living there instead).

Findings

None. No SQL injection was found. No files required a fix.

Verified clean

All six files are fully parameterized (`PDO::ATTR_EMULATE_PREPARES => false` , real prepared statements) and use only two safe patterns wherever a query string is built dynamically:

1. **Bound `?` placeholders for every value** — every `WHERE` /filter/ `LIKE` / `IN (...)` clause built dynamically appends `?` and pushes the real value onto a params array passed to `execute()` . `IN ($ph)` clauses build only the placeholder string from `count($ids)` , never from the values.
2. **Whitelist-checked values, never raw input, wherever a variable is concatenated into SQL as a fragment (column names, table names)** — every such case traces back to a hardcoded PHP array literal or a schema-existence check, never request input: - Dynamic column-name builders (`AdminController.php admin_update_item` , `RideController.php driver_update_vehicle_info` , `ServiceGiverController.php sg_update_profile` , `AuthController.php` 's registration/etelid-login INSERT builders) iterate a hardcoded `$allowed` / `$fields` array — the request body only ever supplies the *value*. - `LIMIT $var` spots are all confirmed `intval()` 'd (often `min()` / `max()` -clamped) before interpolation. - Date-range filters are always a ternary over 2–3 hardcoded SQL fragments; the raw `$days` / `$period` value never reaches the SQL string. - Backtick-quoted identifiers in schema-bootstrap DDL come from hardcoded array literals or server-generated strings, never request input. - No unwhitelisted dynamic `ORDER BY` column/direction was found anywhere in the six files.

Needs Dagi's judgment call

None of the six assigned files needed a fix. One scope note: `admin_get_users` , `admin_get_orders` / `get_orders` , `admin_get_couriers` , `admin_update_courier` actually live as inline `case` blocks in `boot.php` , not in `AdminController.php` despite being admin list/filter endpoints. They were spot-checked here (clean — whitelisted `role` / `status` filters, `intval()` 'd `limit` , bound `?` params throughout) but `boot.php` itself was outside the assigned scope; if a full formal sign-off on `boot.php` is wanted, treat that as a separate pass.

---

3. XSS / output escaping (Section 6)

Checked: `innerHTML` / `outerHTML` / `.html(` / `document.write` / `insertAdjacentHTML` sinks across `delivery/*.html` , `delivery/js/*.js` , `delivery/js/admin/*.js` , `delivery/driver/` , `delivery/ride/` , `delivery/courier/` , `delivery/equb/` , and `delivery/js/releases/**` (~120+ sinks found; ~40 reviewed in depth because they render a user-controlled field the audit specifically called out: recipient name/phone, delivery notes/addresses, chat/support messages, service-giver item name/description including `name_am` / `description_am` , ride/order issue reports).

Fixed (6 real stored-XSS sinks)

All fixed by reusing the codebase's existing escaping helpers (`escapeHtml()` in `js/core.js` , `esc()` in `js/admin-roles.js`) — no new helper needed except one bug fix (see below).

1. **`js/customer.js` — storefront item grid + item-detail modal.** `item.name` / `item.name_am` / `item.description` / `item.description_am` / `item.business_name` (all set by the Service Giver) rendered unescaped. A malicious/compromised Service Giver could plant an XSS payload in a product name and have it fire in every customer's browser viewing that storefront. **Severity: High** — broad customer-facing surface, and exactly the `name_am` / `description_am` fields the audit brief called out. Fixed with `escapeHtml()` ; also hardened the `onclick="addToCart(...)"` JS-string builders by stripping `< > " ' \` (they were breakable as both a JS string and an HTML attribute). Also fixed for defense-in-depth: the customer's own support-message thread and own order-list recipient name (currently self-XSS-only, but one accidental refactor away from showing other people's text).
2. **`js/courier.js` — available-jobs list + accepted-job card.** Customer-entered `recipient_name` , `recipient_phone` , delivery notes/address rendered unescaped — a customer could plant a payload in their delivery address at checkout and have it execute in the courier's app. **Severity: High** (any customer → any courier). Fixed with `escapeHtml()` ; the `navigateToCustomer(...)` onclick string additionally hardened.
3. **`js/servicegiver.js` — incoming-orders list.** `order.order_code` , `from_zone` , `to_zone` , `recipient_name` unescaped (separate from the already-escaped print-slip functions in the same file). **Severity: High.** Fixed with `escapeHtml()` .
4. **`js/admin.js` — four admin tables.** `renderUserTable` , `renderSGTable` , `renderItemTable` , `renderCourierTable` all rendered account-holder-chosen display names (`name` , `phone` , `business_name` , etc. — settable by any customer/courier/ service-giver at signup) unescaped into the admin panel. An admin viewing Users/Service-Givers/Items/Couriers would execute an attacker's payload with the admin's own session. **Severity: High/Critical** — stored XSS targeting the highest-privilege role. Fixed with the existing `esc()` helper. (Note: `renderCourierTable` turned out to be shadowed at runtime by `admin-couriers-premium.js` 's own version of the same function name, which already escapes correctly — fixed the `admin.js` copy anyway for defense-in-depth, since it's a latent trap if the premium override is ever removed.)
5. **`js/driver.js` — ride-history detail modal.** `passenger_name` , `passenger_phone` , `pickup_address` , `dropoff_address` unescaped, while the live active-ride card in the same file correctly escapes the same fields. **Severity: High.** Fixed with the file's existing local `esc()` .
6. **`js/ride.js` — ride-history list + detail modal (rider side, mirror of #5).** `driver_name` , `driver_phone` , `vehicle_plate` , `vehicle_type` , addresses unescaped. **Severity: High.** Fixed with `escapeHtml()` .

Bug found and fixed while doing this work: `admin.js` called `esc()` (and so did a pre-existing, untouched call site at line 211 for order-issue rendering) but `esc()` was never defined anywhere in scope on `admin.html` — `core.js` only defined `escapeHtml()` . This is a real correctness bug, not just a security one: it would throw a `ReferenceError` and break those admin table renders entirely (worse than the XSS gap itself, since it crashes the render instead of just failing to escape). Fixed by adding a global `function esc(str){ return escapeHtml(str); }` alias in `js/core.js` , which loads before every page's own script on every page in this app.

Deploy-path fix: `driver/` , `ride/` , `customer/` , `courier/` app code is actually served via versioned bundles at `js/releases/{app}/{version}/` , built from the unbundled sources by `js/build-bundles.js` (terser, `mangle: {toplevel: false}`). Patching only the unbundled source files would not have reached the live pages. After fixing sources, rebuilt via `node js/build-bundles.js` and synced the output into the currently-live release folders (`js/releases/driver/v6/` , `ride/v5/` , `customer/v1/` , `courier/v1/`). Verified with `node --check` (syntax) and by grepping the minified bundles for the new `escapeHtml(...)` / `esc(...)` calls.

Verified clean

- `js/ride_chat.js` (in-ride chat) — already escapes via a local `esc()` .
- `js/admin-support.js` (Contact-Support inbox) — already wraps in `escapeHtml()` .
- `js/admin-rides.js` / `js/admin.js` ride/order issue panels — already escaped.
- `js/admin/servicegiver-ext.js` — consistently uses `escapeHtml()` already.
- `js/servicegiver.js` print-slip builders — already escaped (the Bluetooth-printer byte builder uses the same raw fields but writes to a physical printer, not the DOM — not an XSS sink).
- `js/ride.js` / `js/driver.js` live active-ride cards — set via `.textContent` , safe by construction.
- `equb/index.html` / `js/admin-equb.js` — already escaped; equb data is founder/admin authored anyway, not general customer free text.

Needs Dagi's judgment call

- **`order_code` / zone names** left unescaped in a few remaining spots — these are server-generated or drawn from a fixed admin-curated list today, so low/no risk, but would need the same treatment if either ever becomes user-editable free text. Not fixed now to avoid masking a future real gap behind blanket "escape everything."
- **`item.image_url` / `delivery_photo_url` / map-thumbnail URLs** are placed raw into `src="..."` attributes. These come from an upload flow — whether this is exploitable depends on whether the upload endpoint already constrains the value to a same-origin/ CDN path (moot) or accepts arbitrary strings (needs attribute escaping + a URL-scheme allowlist, since `javascript:` -style payloads are a separate concern from HTML injection). Left for Dagi's call on the upload endpoint's actual constraints.

- **`js/releases/driver/v6/` appears to be missing several files** (`driver-bundle-extra.js`, `driver-rate-rider.js`, `driver-announcements-v2.js`, `sos-button.js`) that `release-loader.js`'s manifest expects to find there. This looks like a pre-existing gap unrelated to XSS (an incomplete v6 release), not something this pass introduced — flagging in case it's silently causing the loader to fall back to an older version for those scripts in production.

---

4. Session/cookie security (Section 5.3)

Checked: every `setcookie(` / `session_set_cookie_params(` call under `delivery/`; `boot.php`'s `gen_token()` / `verify_token()` / `get_auth_user()` / `require_auth()` / `get_bearer()` / `etel_bump_token_version()`; every `refresh`-named action; whether `EtelID_Auth::create_session()` exists in this repo.

Findings

`EtelID_Auth::create_session()` is out of scope — it lives in the separate EtelID WordPress plugin (`EtelId/includes/class-etelid-auth.php`), not in the Delivery & Ride codebase this task covers. Delivery's own auth (`gen_token()` / `verify_token()` in `boot.php`) is a completely separate custom bearer-token scheme.

Cookies: only one place in `delivery/` sets a cookie, and it is already correctly hardened. `grep -rn "setcookie(" delivery/` returns zero results. The only cookie-adjacent code is `delivery/api/etelid_auth.php` (a client library bridging Delivery to EtelID's browser-based SSO), which calls `session_set_cookie_params()` with `'secure' => $https` (conditional on the real request scheme, correctly handling a forwarded-proto setup), `'httponly' => true`, `'samesite' => 'Lax'`, before `session_start()`. It also enforces an 8-hour inactivity timeout (`time() - $_SESSION['etelid_ts'] > 28800`). **No change was needed here** — already correct.

The rest of the API (the large majority) is a pure bearer-token API with no cookie-based session at all. Cookie flags are not applicable there because there is no cookie — stated explicitly rather than fabricating a mechanism that doesn't exist.

Token lifetime/expiry/renewal, documented from the literal code: - `gen_token($user_id, $version)` packs `user_id:issued_at:token_version:hmac_sha256(...)`, base64-encoded, HMAC-SHA256-signed with `TOKEN_SECRET_FINAL`. - `verify_token()` enforces a **hard 7-day lifetime** (`(time() - intval($ts)) > 86400 * 7` → reject) and rejects malformed/old-format tokens closed, not open. Signature check uses `hash_equals()` (timing-safe). - `get_auth_user()` additionally compares the token's embedded `token_version` against the user's live `token_version` column — a mismatch invalidates the token immediately, independent of the 7-day window. - **On expiry: a clean 401, no silent refresh.** No token-refresh action exists anywhere in the codebase (grepped) — expiry always means "log in again." - **Logout genuinely revokes.** `logout` calls `etel_bump_token_version()` (`UPDATE users SET token_version = token_version + 1`), which immediately invalidates every outstanding token for that user, on every device — not just the one that called logout. Password-change and password-reset both also bump the version on success.

Verified clean

- No raw `setcookie()` anywhere in `delivery/`.
- `etelid_auth.php`'s one real cookie usage already has Secure/HttpOnly/SameSite correct.
- Token expiry is a real, enforced 7-day hard cap.
- Logout performs real server-side revocation, not just a client-side "forget it."
- No silent/automatic token refresh exists to paper over a revoked/expired credential.

Needs Dagi's judgment call

A note on `delivery/api/audit_agent.php`: this is an internal, non-submitted dev tool (a one-click "ship this project's own source to an AI reviewer" self-audit button) whose embedded system prompt includes calibration rules like *"`session_set_cookie_params([...])` is CORRECT and secure. Flag as passed, not as issue"* and similar pre-declared-safe patterns for SQLi and rate-limiting. Read in context, this is anti-false-positive grading guidance for an internal tool (preventing a generic LLM reviewer from flagging the project's own already-safe, already-established patterns) — not an attempt to game the actual INSA submission, since this tool's output is never part of the docx package or submitted anywhere. The cookie flags it pre-declares as "correct" were independently verified above on their own merits, not because the tool said so. Still, it's worth Dagi's attention: an internal review tool that pre-grades its own house patterns as "passed" is a design choice worth a second look, especially if it's ever used for anything beyond internal iteration.

---

5. Error handling and logging sweep (Section 5.4)

Checked: all 62 `.php` files under `delivery/` for `catch` blocks with a genuinely empty body or a comment-only body. ~60 such blocks found; the large majority are an established, intentional, already-commented soft-fail pattern in this codebase ("table may not exist yet", "non-fatal", "best-effort" schema-

migration guards) and were correctly left untouched — flagging deliberate fallbacks would be noise, not signal.

Fixed

Added `error_log('[ETEL] ' . <context> . ': ' . $e->getMessage());` to **9 catch blocks across 5 files**, where the swallowed exception guarded something non-trivial (a real DB write, an audit-log trail, brute-force protection, or dispatch/financial logic) with no existing comment indicating the failure was expected. Log-only — no control flow changed, matching the pattern already established elsewhere in these same files.

File	Guarded operation	Why it matters
`api/updater-api.php` (`log_updater_action()`)	Audit-log INSERT for admin file-deploy actions	This *is* the security audit trail for the production updater tool — a silent failure meant a deploy could go completely untraced.
`api/boot.php` (`get_or_create_courier_profile()`)	`INSERT IGNORE INTO couriers` auto-provisioning	A real INSERT failure silently returned `null` with no way to distinguish "no such courier" from "DB error."
`api/boot.php` (`rate_limit_fail()`)	`INSERT INTO login_attempts`	A silent write failure here means failed logins stop being counted — undermines brute-force lockout invisibly.
`api/boot.php` (`rate_limit_clear()`)	`DELETE FROM login_attempts` on success	Consistency fix; lower severity (worst case: user stays rate-limited a bit longer).
`api/boot.php` (`admin_dashboard_stats`)	Admin financial-dashboard `SELECT COUNT` / `SUM(fare_total)` queries	A genuine query failure silently zeroed the whole stats block — an admin could misread "zero revenue" as real instead of a broken query.
`api/boot.php` (`admin_create_service_giver`)	Newly-created service giver's lat/lng UPDATE	Minor (optional field) but a real UPDATE whose failure was previously invisible.
`api/lib/delivery_order_core.php` (auto-dispatch notify)	Courier push/in-app notification after auto-assignment	If this silently fails, the assigned courier never finds out, with zero server-side trace.
`api/lib/delivery_order_core.php` (auto-dispatch outer catch)	The whole nearest-courier auto-dispatch routine	A real order-fulfillment failure path (order silently stays unassigned) that previously left no log at all.
`api/controllers/AuthController.php` (`change_password` token refetch)	Re-reading `token_version` after a password change to issue a working replacement token	Silent failure means the response token is built with a stale version and fails the very next request's version check — a "why am I logged out right after changing my password" bug with no trail.
`api/controllers/AuthController.php` (service-giver login lookup)	`SELECT ... FROM service_givers WHERE user_id=?` during login	A real query failure previously looked identical to "this account has no service-giver row" (generic 404).

Incidental ASCII-safety fix: while editing `boot.php`, the mandatory ASCII scan turned up 8 pre-existing non-ASCII middle-dot (`·`) separators in runtime string literals (not comments) in demo/seed-data diagnostic messages. Normalized to `-` (matching the ASCII style already used a few lines below in the same file) since the file was already being touched and the ASCII constraint is a hard requirement — pure character substitution, no logic change.

All 5 edited files verified with `php -l` and the ASCII check.

Verified clean

Every other empty/comment-only catch (schema-migration `ADD COLUMN` guards, "table may not exist yet" reads, cosmetic fallbacks across `RideController.php`, `OrderController.php`, `ride_fare.php`, `delivery_pricing.php`, `AdminController.php`, `ServiceGiverController.php`, `AgentController.php`, `migrate.php`, `telegram_webhook.php`) is an established, already-documented, intentional pattern — correctly left untouched.

Needs Dagi's judgment call

`AuthController.php`'s `change_password` token-refetch failure is now logged but still returns a potentially-stale token. Whether that should hard-fail the whole `change_password` call (forcing explicit re-login) instead of the current soft-degrade is a product decision, not something changed unilaterally here.

---

6. Input validation audit (Section 5.2)

Checked: all 15 controllers under `delivery/api/controllers/` for how incoming `$_POST` / `$_GET` /JSON-body values are validated before reaching SQL or a filesystem operation, plus the shared `delivery/api/validate.php` helpers. Special focus on the delivery-photo upload path (`$_FILES`,

`move_uploaded_file`, destination handling).

Findings

General pattern is strong and consistent — no fix required. Zero instances found of a raw `$_GET` / `$_POST` /body value concatenated directly into a SQL string. The only places a variable is interpolated directly into a query string are table/column names, and every one of those traces back to a fixed, hardcoded internal array — never user input. Numeric IDs are consistently `intval()`'d before use; `validate.php` centralizes phone (`etel_validate_phone()`) and amount (`etel_validate_amount()`, bounded/finite/ range-checked) validation.

Delivery-photo / general image upload (`delivery/api/upload.php`) — already well-hardened: - **Content-sniffed MIME type** (`mime_content_type($file['tmp_name'])`, inspects real bytes) against a hard whitelist (`image/jpeg`, `image/png`, `image/gif`, `image/webp`) — a renamed `.php` file with a spoofed `Content-Type` header is still rejected, since the client's declared type is never trusted. - **5 MB size cap** (8 MB on the separate admin branding-upload path) before any processing. - **No path-traversal exposure**: destination folder comes from a fixed `$type_map` whitelist; an unrecognized `type` falls back to a default folder rather than becoming part of the path. The filename is generated entirely server-side (`$prefix . $user['id'] . '_' . time() . '_' . bin2hex(random_bytes(4)) . '.' . $ext`) — the client's original filename never reaches the destination path. - Bearer-token auth required before any file is processed. - The admin branding-upload and admin file-deploy (`updater.php` / `updater-api.php`) paths similarly whitelist destinations via a hardcoded map; the deploy tool has a dedicated `etel_safe_rel_path()` / `etel_canonicalize()` pair resolving `.` / `..` segments in one pass (documented as a fix for an older, bypassable single-pass filter).

No genuine unvalidated-input-into-SQL-or-filesystem gap found in any of the 15 controllers.

Verified clean

- No raw request value reaches a SQL query string unparameterized anywhere in `delivery/api/`.
- No raw request value reaches a filesystem path without a whitelist and/or canonicalization first.
- Delivery-photo upload specifically: content-sniffed MIME check, size cap, whitelisted destination, fully server-generated filename.

Needs Dagi's judgment call

None requiring a fix. Optional defense-in-depth: `push_subscriptions.endpoint` / `p256dh` have no explicit max-length check before insert (would only turn a DB-truncation exception into a friendlier 400 — not itself a security gap).

---

7. Known open items — confirmed current status

7.1 `backups/` directory web-accessibility

Checked: the whole `delivery/` tree for any `backups/` directory. None is checked into git, but `delivery/api/updater.php` (`BACKUP_DIR = __DIR__ . '/backups/'`) creates `delivery/api/backups/` at runtime on first use of its "backup" action, copying source files (`index.html`, `config.js`, `etel-engine.js`, `sw.js`, `skills/registry.json`) into timestamped snapshots.

Found: `delivery/api/.htaccess` (covering all of `api/`) only sets `Options -Indexes` — no explicit deny rule scoped to `backups/`, and no `.htaccess` existed inside the directory itself (it didn't exist in the repo checkout). This was a real gap: a direct request to `api/backups/<timestamp>/config.js` was not explicitly denied by anything but directory-listing suppression.

Fixed: created `delivery/api/backups/.htaccess`:

```
<IfModule mod_authz_core.c>
    Require all denied
</IfModule>
<IfModule !mod_authz_core.c>
    Order allow,deny
    Deny from all
</IfModule>
Options -Indexes
```

Matches the Apache 2.4+ primary rule with an older-Apache fallback, mirroring the only other `backups/`-lockdown precedent in the wider Etel Group codebase (`EtelAi/backups/.htaccess`).

7.2 Hardcoded admin password in the Updater

Checked: `delivery/api/updater.php` (296 lines, full read) and `delivery/api/updater-api.php` (the actual admin-panel updater wired into `admin.html`'s Updater tab), against `boot.php`'s `etel_env()`/fail-closed reference pattern.

Found — already fixed in a prior session, not hardcoded: - `updater.php` loads `UPDATER_PASSWORD` from `api/.env` via its own self-contained line-parser (functionally equivalent to `boot.php`'s `etel_env()`), with a `getenv('UPDATER_PASSWORD')` fallback. If unset, it fails closed — 503 + `"UPDATER_PASSWORD not set in api/.env - refusing to start for safety."` — no default password, no silent bypass. `api/.env.example` documents the placeholder to replace. - `updater-api.php` (the more powerful, DB-backed updater actually used day-to-day) loads its DB/token config the same layered way (`.env` → `getenv()` → constant → default), fails closed (503) if `DB_USER`/`DB_PASS` are unset, and authenticates via the same bearer-token mechanism as the rest of the app — not a separate hardcoded password at all. Its CORS origin allowlist is also scoped to `https://ethelmarket.com`/`https://www.ethelmarket.com` rather than a wildcard.

No changes were needed for this item.

---

8. Full API endpoint inventory (Section 8.3)

All actions are dispatched by `delivery/api/boot.php`'s single entrypoint. The action name is read as `$body['action']` (POST JSON body) **or** `$_GET['action']` — every action is reachable via either at the router level. The one real GET-only carve-out: `boot.php` accepts the bearer token via `?token=` (instead of the `Authorization` header) only for action names starting with `sse_` or ending in `_beacon_ping` (`EventSource`/`navigator.sendBeacon` can't set custom headers) — those rows are marked **GET** below; everywhere else "POST" means "POST (GET also technically accepted by the router, but the real frontends always POST)".

Access level is determined by the actual guard call in the handling code: `require_auth()` (any logged-in role), `require_admin($pdo,'capability')` (role `admin` + that capability), `require_courier()` (role `courier` or `admin`), a manual `$u['role'] !== 'X'` check, or the shared `ETELID_SERVICE_KEY` service-key check (internal system-to-system calls).

8.1 `boot.php` — inline actions

Action	Method	Access Level
`admin_purge_cache`	POST	Admin (any)
`get_my_notifications`	POST	Authenticated — any role
`ping`, `db_ping`	POST	Public
`version_check`	POST	Public
`logout`	POST	Public (optional auth — no-ops cleanly with no/invalid token)
`get_config`	POST	Public (returns Google Maps API key)
`get_patch_info`	POST	Authenticated — any role
`get_profile`, `my_profile`	POST	Authenticated — any role
`get_zones`	POST	Public
`get_delivery_tiers`	POST	Public
`get_bank_accounts`	POST	Public
`admin_get_service_givers`, `admin_get_service_givers_list`	POST	Admin (any)
`admin_get_users`	POST	Admin (any)
`admin_get_couriers`	POST	Admin (any)
`admin_update_courier`	POST	Admin (any)
`admin_get_live_couriers`	POST	Admin (any)
`admin_get_orders`, `get_orders`	POST	Admin (any)
`admin_dashboard_stats`	POST	Admin (any)
`admin_verify_service_giver`	POST	Admin (any)
`admin_suspend_service_giver`	POST	Admin (any)
`admin_create_service_giver`	POST	Admin (any)
`get_items`, `customer_get_items`	POST	Public (item/menu catalog)
`etelid_members_list`	POST/GET	Internal — `ETELID_SERVICE_KEY` header

Action	Method	Access Level
`etelid_member_detail`	POST/GET	Internal — `ETELID_SERVICE_KEY` (header or `?key=`)
`sse_admin_delivery_system_test`	**GET** (SSE, `?token=`)	Admin, `orders.manage`
`sse_admin_system_test`	**GET** (SSE, `?token=`)	Admin, `rides.manage`
`sse_ride_status`	**GET** (SSE, `?token=`)	Authenticated (scoped: ride's own passenger)
`sse_driver_stream`	**GET** (SSE, `?token=`)	Authenticated — any role
`reset_request`, `send_reset_otp`	POST	Public (password-reset step 1)
`reset_verify`, `verify_reset_otp`	POST	Public (password-reset step 2)
`reset_password`	POST	Public (password-reset step 3)

8.2 `controllers/AuthController.php`

Action	Method	Access Level
`register`	POST	Public
`login`	POST	Public
`sg_login`	POST	Public
`etelid_login`	POST	Public (EtelID SSO exchange)
`admin_etelid_login`	POST	Public (EtelID SSO exchange, staff path)
`change_password`	POST	Authenticated — any role

8.3 `controllers/AdminController.php` (require_admin + named capability on every action except `get_feature_flags`)

Action	Access Level (capability)
`get_feature_flags`	Public
`admin_list_etelpay_overrides`	Admin, `owner.only`
`admin_set_etelpay_override`	Admin, `owner.only`
`admin_remove_etelpay_override`	Admin, `owner.only`
`admin_list_support_threads`	Admin, `orders.view`
`admin_get_support_thread`	Admin, `orders.view`
`admin_reply_support_message`	Admin, `orders.view`
`admin_toggle_feature`	Admin, `owner.only`
`admin_list_release_channels`	Admin, `owner.only`
`admin_set_release_channel`	Admin, `owner.only`
`admin_promote_canary_to_stable`	Admin, `owner.only`
`admin_rollback_stable`	Admin, `owner.only`
`admin_add_canary_tester`	Admin, `owner.only`
`admin_remove_canary_tester`	Admin, `owner.only`
`admin_search_canary_candidates`	Admin, `owner.only`
`admin_run_code_audit`	Admin, `owner.only`
`admin_get_audit_settings`	Admin, `owner.only`
`admin_save_audit_settings`	Admin, `owner.only`
`admin_create_user`	Admin, `users.manage`
`admin_assign_service_giver`	Admin, `service_givers.manage`
`admin_add_item`	Admin, `items.manage`
`admin_get_all_items`	Admin, `items.view`

Action	Access Level (capability)
admin_update_item	Admin, items.manage
admin_delete_item	Admin, items.manage
admin_toggle_item	Admin, items.manage
get_service_givers	Admin, service_givers.view
admin_get_service_giver	Admin, service_givers.view
admin_update_order	Admin, orders.manage
admin_get_order_receipt	Admin, orders.view
admin_get_service_giver_statistics	Admin, service_givers.view
admin_get_confirmations	Admin, orders.view
admin_backfill_order_service_giver_ids	Admin, orders.manage
admin_force_confirm	Admin, orders.manage
admin_analytics	Admin, analytics.view
admin_backfill_cash_commission	Admin, finance.manage
admin_topup_etel_money	Admin, finance.manage
admin_get_pending_etel_topups	Admin, finance.view
admin_approve_etel_topup	Admin, finance.manage
admin_reject_etel_topup	Admin, finance.manage
admin_ride_growth	Admin, analytics.view
admin_get_ride_fare_rules	Admin, pricing.manage
admin_update_ride_fare_rule	Admin, pricing.manage
admin_add_vehicle_type	Admin, pricing.manage
admin_delete_vehicle_type	Admin, pricing.manage
admin_update_ride_surge_multiplier	Admin, pricing.manage
admin_get_goal_tiers	Admin, pricing.manage
admin_save_goal_tier	Admin, pricing.manage
admin_delete_goal_tier	Admin, pricing.manage
admin_run_system_test	Admin, rides.manage
admin_get_ride_peak_windows	Admin, pricing.manage
admin_update_ride_peak_windows	Admin, pricing.manage
admin_get_zones	Admin, settings.view
admin_update_zone	Admin, settings.manage
admin_add_zone	Admin, settings.manage
admin_get_delivery_tiers	Admin, settings.view
admin_update_delivery_tier	Admin, settings.manage
admin_get_bank_accounts	Admin, finance.view
admin_update_bank_account	Admin, finance.manage
admin_create_announcement	Admin, settings.manage
admin_get_announcements	Admin, settings.view
admin_delete_announcement	Admin, settings.manage
admin_get_launch_checklist_state	Admin, settings.view
admin_toggle_launch_checklist_item	Admin, settings.manage
admin_get_ride_issues	Admin, rides.view

Action	Access Level (capability)
`admin_resolve_ride_issue`	Admin, `rides.manage`
`admin_get_order_issues`	Admin, `orders.view`
`admin_resolve_order_issue`	Admin, `orders.manage`
`admin_get_sos_alerts`	Admin, `sos.view`
`admin_resolve_sos`	Admin, `sos.manage`
`admin_delete_zone`	Admin, `settings.manage`
`admin_list_role_permissions`	Admin, `owner.only`
`admin_update_role_permissions`	Admin, `owner.only`
`admin_delete_role_permissions`	Admin, `owner.only`
`admin_get_my_capabilities`	Admin (any)

(All: POST, GET also accepted by router.)

8.4 `controllers/EquityController.php` (founder financial/ownership data)

Action	Access Level
`equity_get_settings`	Admin, `equity.view`
`equity_update_settings`	Admin, `equity.manage`
`equity_list_investors`	Admin, `equity.view`
`equity_link_investor_user`	Admin, `equity.manage`
`equity_unlink_investor_user`	Admin, `equity.manage`
`investor_portal_am_i_investor`	Authenticated — any role
`investor_portal_get_my_view`	Authenticated (scoped: linked to an investor row)
`investor_portal_get_my_growth_planner_data`	Authenticated (same scoping)
`equity_add_investor`	Admin, `equity.manage`
`equity_update_investor`	Admin, `equity.manage`
`equity_regenerate_investor_token`	Admin, `equity.manage`
`equity_delete_investor`	Admin, `equity.manage`
`equity_get_dashboard`	Admin, `equity.view`
`equity_preview_distribution`	Admin, `equity.view`
`equity_distribute_profit`	Admin, `equity.manage`
`equity_list_distributions`	Admin, `equity.view`
`equity_get_distribution`	Admin, `equity.view`
`equity_export_investors_csv`	Admin, `equity.view`

8.5 `controllers/InvestorPortalController.php` (legacy token-link portal, NOT admin-gated)

Action	Access Level
`investor_portal_get_view`	Public — gated by possession of a personal investor token in the request, not a login
`investor_portal_get_growth_planner_data`	Public — same personal-token gating

8.6 `controllers/SystemResetController.php` (full production data-wipe — DANGER ZONE)

Action	Access Level
`system_reset_preview`	Admin, `system.reset` (not included by a generic Owner grant — must be added explicitly)
`system_reset_backup`	Admin, `system.reset`

Action	Access Level
`system_reset_execute`	Admin, `system.reset`

8.7 `controllers/EqubController.php`

Action	Access Level
`equb_am_i_permitted`	Authenticated — any role
`equb_create_group`	Admin, `equb.manage`
`equb_join_group`	Authenticated — any role
`equb_leave_group`	Authenticated — any role
`equb_start_group`	Admin, `equb.manage`
`equb_get_my_groups`	Authenticated — any role
`equb_get_group_detail`	Authenticated — any role
`equb_admin_create_package`	Admin, `equb.manage`
`equb_admin_update_package`	Admin, `equb.manage`
`equb_admin_list_packages`	Admin, `equb.view`
`equb_list_packages`	Authenticated — any role
`equb_select_package`	Authenticated — any role
`equb_admin_add_member`	Admin, `equb.manage`
`equb_list_open_groups`	Authenticated — any role
`equb_admin_grant_access`	Admin, `equb.manage`
`equb_admin_revoke_access`	Admin, `equb.manage`
`equb_admin_list_participants`	Admin, `equb.view`
`equb_admin_force_process_round`	Admin, `equb.manage`
`equb_admin_list_groups`	Admin, `equb.view`

8.8 `controllers/PlanningController.php`

Action	Access Level
`planning_list_hires`	Admin, `planning.view`
`planning_add_hire`	Admin, `planning.manage`
`planning_update_hire`	Admin, `planning.manage`
`planning_delete_hire`	Admin, `planning.manage`
`planning_get_notes`	Admin, `planning.view`
`planning_save_notes`	Admin, `planning.manage`
`planning_reset_and_seed_demo`	Admin, `planning.manage`

8.9 `controllers/PlatformEconomicsController.php`

Action	Access Level
`platform_economics_get_costs`	Admin, `platform_economics.view`
`platform_economics_update_costs`	Admin, `platform_economics.manage`
`platform_economics_get_growth_assumptions`	Admin, `platform_economics.view`
`platform_economics_save_growth_assumptions`	Admin, `platform_economics.manage`
`platform_economics_reset_growth_assumptions`	Admin, `platform_economics.manage`
`platform_economics_get_summary`	Admin, `platform_economics.view`

8.10 `controllers/DriverFuelController.php` (module-level require_auth; data scoped to caller's own rows)

Action	Access Level
`driver_add_fuel_log`	Authenticated — any role
`driver_list_fuel_logs`	Authenticated — any role
`driver_delete_fuel_log`	Authenticated — any role
`driver_get_fuel_summary`	Authenticated — any role

8.11 `controllers/CourierFuelController.php` (same pattern as 8.10)

Action	Access Level
`courier_add_fuel_log`	Authenticated — any role
`courier_list_fuel_logs`	Authenticated — any role
`courier_delete_fuel_log`	Authenticated — any role
`courier_get_fuel_summary`	Authenticated — any role

8.12 `controllers/PushController.php` (module-level require_auth)

Action	Access Level
`save_push_subscription`	Authenticated — any role
`remove_push_subscription`	Authenticated — any role
`test_send_push`	Authenticated — any role

8.13 `controllers/AgentController.php` (EtelAI booking connector — every action calls `agent_verify_service_key()` first)

Action	Access Level
`agent_find_and_quote`	Internal — `ETELID_SERVICE_KEY` header
`agent_search_items`	Internal — `ETELID_SERVICE_KEY` header
`agent_order_quote`	Internal — `ETELID_SERVICE_KEY` header
`agent_place_order`	Internal — `ETELID_SERVICE_KEY` header
`agent_ride_quote`	Internal — `ETELID_SERVICE_KEY` header
`agent_book_ride`	Internal — `ETELID_SERVICE_KEY` header
`agent_ride_status`	Internal — `ETELID_SERVICE_KEY` header
`agent_cancel_ride`	Internal — `ETELID_SERVICE_KEY` header
`agent_order_status`	Internal — `ETELID_SERVICE_KEY` header

8.14 `controllers/OrderController.php`

Action	Access Level
`customer_create_order`	Authenticated (business rule further restricts to `customer` / `admin` inside `delivery_execute_order_creation()`)
`customer_get_orders`	Authenticated (scoped to caller's own orders)
`submit_order_issue`	Authenticated — any role
`customer_get_order_issues`	Authenticated — any role
`courier_get_available_orders`	Courier or admin
`courier_accept_order`	Courier or admin
`courier_update_order_status`	Courier or admin
`courier_get_profile`	Courier or admin

Action	Access Level
`courier_go_online`	Courier or admin
`courier_update_location`	Courier or admin
`courier_go_offline`	Courier or admin
`courier_get_kpi_summary`	Courier or admin
`courier_get_my_orders`	Courier or admin
`rate_order`	Authenticated — any role
`get_order_ratings`	Admin, `orders.view`
`track_order`	Authenticated — any role
`schedule_order`	Authenticated — any role
`get_scheduled_orders`	Authenticated — any role
`request_pickup_confirmation`	Courier or admin
`check_pickup_confirmation`	Courier or admin (ownership-checked — see Section 1 fix)
`sg_pending_confirmations`	Role `service_giver` or `admin`
`sg_confirm_pickup`	Role `service_giver` or `admin`
`request_delivery_confirmation`	Courier or admin
`check_delivery_confirmation`	Courier or admin (ownership-checked — see Section 1 fix)
`customer_pending_confirmations`	Authenticated — any role
`customer_confirm_delivery`	Authenticated — any role
`customer_wallet_history`	Authenticated — any role
`customer_get_favorites`	Authenticated — any role
`customer_toggle_favorite`	Authenticated — any role
`customer_get_addresses`	Authenticated — any role
`customer_add_address`	Authenticated — any role
`customer_update_address`	Authenticated — any role
`customer_delete_address`	Authenticated — any role
`customer_get_messages`	Authenticated — any role
`customer_send_message`	Authenticated — any role
`customer_get_referral_info`	Authenticated — any role
`customer_get_settings`	Authenticated — any role
`customer_update_settings`	Authenticated — any role
`initiate_telebirr_payment`	Authenticated — any role
`check_telebirr_payment`	Authenticated — any role
`admin_list_telebirr_payments`	Role `admin`
`admin_confirm_telebirr_payment`	Role `admin`
`confirm_bank_payment`	Authenticated (customer submits own claimed transfer)
`admin_list_bank_payments`	Role `admin`
`admin_confirm_bank_payment`	Role `admin`
`admin_financial_report`	Role `admin` AND capability `finance.view`
`get_payment_method_logos`	Authenticated — any role
`admin_set_payment_logo`	Role `admin`

8.15 `controllers/ServiceGiverController.php`

Action	Access Level
`sg_profile`	Role `service_giver` or `admin`
`sg_update_profile`	Role `service_giver` or `admin`
`sg_orders`	Role `service_giver` or `admin`
`sg_accept_order`	Role `service_giver` or `admin`
`sg_get_kitchen_ticket`	Role `service_giver` or `admin`
`sg_update_order_status`	Role `service_giver` or `admin`
`sg_order_detail`	Role `service_giver` or `admin`
`sg_scan_qr`	Role `service_giver` or `admin`
`sg_complete_order`	Role `service_giver` or `admin`
`sg_stats`	Role `service_giver` or `admin`
`sg_set_online`	Role `service_giver` or `admin`
`sg_notifications`	Role `service_giver` or `admin`
`sg_add_item`	Role `service_giver` or `admin`
`sg_get_my_items`	Role `service_giver` or `admin`
`sg_update_item`	Role `service_giver` or `admin`
`sg_delete_item`	Role `service_giver` or `admin`
`sg_get_staff`	Role `service_giver` or `admin` (owner-tier for staff-management sub-actions)
`sg_add_staff`	Role `service_giver` or `admin` , owner-tier within the business
`sg_remove_staff`	Role `service_giver` or `admin` , owner-tier
`sg_update_staff_role`	Role `service_giver` or `admin` , owner-tier
`sg_list_roles`	Role `service_giver` or `admin`
`sg_save_role`	Role `service_giver` or `admin` , owner-tier
`sg_delete_role`	Role `service_giver` or `admin` , owner-tier
`sg_my_role`	Role `service_giver` or `admin`
`sg_financial_report`	Role `service_giver` or `admin`
`sg_get_pending_payout_request`	Role `service_giver` or `admin`
`admin_list_payout_requests`	Role `admin`
`admin_review_payout_request`	Role `admin`

8.16 `controllers/RideController.php` (module-level require_auth — every action needs a valid token at minimum; see Section 1 for the driver-privilege gap)

Action	Access Level
`ride_get_vehicle_types`	Authenticated — any role
`ride_estimate_fare`	Authenticated — any role
`passenger_get_nearby_drivers`	Authenticated — any role
`book_ride`	Authenticated — any role
`get_etelpay_merchant`	Authenticated — any role
`get_ride_path`	Authenticated — any role
`ride_status`	Authenticated — any role
`my_active_ride`	Authenticated — any role
`cancel_ride`	Authenticated — any role
`rate_ride`	Authenticated — any role

Action	Access Level
`rate_rider`	Authenticated — any role
`ride_get_share_link`	Authenticated — any role
`ride_stop_sharing`	Authenticated — any role
`ride_schedule`	Authenticated — any role
`ride_get_scheduled`	Authenticated — any role
`ride_cancel_scheduled`	Authenticated — any role
`ride_history`	Authenticated — any role
`driver_go_online`	Authenticated — any role *(see Section 1 judgment call)*
`driver_go_offline`	Authenticated — any role
`driver_update_location`	Authenticated — any role
`driver_beacon_ping`	**GET** (`?token=`) — Authenticated
`admin_get_rides`	Role `admin` + `rides.view`
`admin_get_ride_stats`	Role `admin` + `rides.view`
`admin_cancel_ride`	Role `admin` + `rides.manage`
`admin_get_drivers`	Role `admin` + `users.view`
`admin_get_live_drivers`	Role `admin` + `users.view`
`admin_get_suspended_drivers`	Role `admin` + `users.view`
`admin_suspend_driver`	Role `admin` + `users.manage`
`admin_unsuspend_driver`	Role `admin` + `users.manage`
`admin_get_flagged_identities`	Role `admin` + `users.manage`
`admin_approve_driver_identity`	Role `admin` + `users.manage`
`admin_reject_driver_identity`	Role `admin` + `users.manage`
`driver_get_nearby_requests`	Authenticated — any role *(see Section 1)*
`driver_get_goal_tiers`	Authenticated — any role
`driver_get_profile`	Authenticated — any role
`driver_get_earnings_for_date`	Authenticated — any role
`driver_get_kpi_summary`	Authenticated — any role
`driver_get_announcements`	Authenticated — any role
`driver_mark_announcement_read`	Authenticated — any role
`driver_get_referral_info`	Authenticated — any role
`submit_ride_issue`	Authenticated — any role
`sos_trigger`	Authenticated — any role
`driver_cashout`	Authenticated — any role *(see Section 1)*
`driver_topup_etel_money`	Authenticated — any role *(see Section 1)*
`driver_submit_manual_topup`	Authenticated — any role
`driver_get_topup_history`	Authenticated — any role
`driver_update_profile`	Authenticated — any role
`driver_get_pending_vehicle_change`	Authenticated — any role
`admin_get_pending_vehicle_changes`	Role `admin` + `rides.view`
`admin_approve_vehicle_change`	Role `admin` + `rides.manage`
`admin_reject_vehicle_change`	Role `admin` + `rides.manage`
`driver_accept_ride`	Authenticated — any role *(see Section 1)*

Action	Access Level
`driver_hail_customer`	Authenticated — any role *(see Section 1)*
`driver_start_ride`	Authenticated — any role *(see Section 1)*
`driver_toggle_wait`	Authenticated — any role
`driver_arrived`	Authenticated — any role *(see Section 1)*
`customer_confirm_arrival`	Authenticated — any role
`driver_complete_ride`	Authenticated — any role *(see Section 1)*
`driver_cancel_ride`	Authenticated — any role
`passenger_get_ride_history`	Authenticated — any role
`send_ride_message`	Authenticated (scoped: ride participant or admin)
`ping_ride_typing`	Authenticated (same scoping)
`get_ride_messages`	Authenticated (same scoping)
`ride_get_favorite_routes`	Authenticated — any role
`ride_add_favorite_route`	Authenticated — any role
`ride_delete_favorite_route`	Authenticated — any role
`ride_get_messages`	Authenticated — any role

Grand total: ~325 distinct action names across `boot.php` (31 inline) + `AuthController.php` (6) + `AdminController.php` (73) + `EquityController.php` (18) + `InvestorPortalController.php` (2) + `SystemResetController.php` (3) + `EqubController.php` (19) + `PlanningController.php` (7) + `PlatformEconomicsController.php` (6) + `DriverFuelController.php` (4) + `CourierFuelController.php` (4) + `PushController.php` (3) + `AgentController.php` (9) + `OrderController.php` (47) + `ServiceGiverController.php` (28) + `RideController.php` (65).

---

9. Sample request/response pairs

All four are based directly on the actual controller code (field names/types exactly as read via `$body['x']` and returned via `boot.php`'s `ok($data)` helper, which wraps every successful response as `{"success": true, "data": {...}, "v": "<APP_VER>"}`, and `fail($msg, $code)`, which returns `{"success": false, "message": "...", "v": "<APP_VER>"}` with the given HTTP status code). Placeholder values below are fictitious — no real customer data.

9.1 Order creation — `customer_create_order`

(`OrderController.php` → `delivery_execute_order_creation()` in `lib/delivery_order_core.php`)

Request (POST, `Authorization: Bearer <token>`, role must be `customer` or `admin`):

```
{
  "action": "customer_create_order",
  "items": [
    { "item_id": 42, "quantity": 2 },
    { "item_id": 57, "quantity": 1 }
  ],
  "to_zone": "Tabor",
  "recipient_name": "Test User",
  "recipient_phone": "0911000000",
  "notes": "Leave at the gate, call on arrival",
  "payment_method": "cash",
  "delivery_tier": "moto",
  "delivery_lat": 7.0650,
  "delivery_lng": 38.4850
}
```

Success response:

```
{
  "success": true,
  "data": {
    "order_id": 10321,
    "order_code": "ORD-10321",
    "status": "pending",
    "service_giver_id": 14,
    "subtotal": 255.00,
    "delivery_tier": "moto",
    "delivery_photo_url": null,
    "delivery_fee": 40.00,
    "service_charge": 5.00,
    "total": 300.00,
    "payment_method": "cash",
    "items_count": 2,
    "message": "Order created and assigned to service giver"
  },
  "v": "3.7.4"
}
```

(`order_code`, `delivery_fee`, `service_charge` depend on live zone/tier pricing — field names are exact, values are illustrative.)

9.2 Ride request — `book_ride`

(`RideController.php`, requires `Authorization: Bearer <token>`, any authenticated role)

Request:

```
{
  "action": "book_ride",
  "pickup_lat": 7.0500,
  "pickup_lng": 38.4700,
  "dropoff_lat": 7.0650,
  "dropoff_lng": 38.4850,
  "pickup_addr": "Mehal Ketema, near the main square",
  "dropoff_addr": "Tabor, Etel Group office",
  "payment_method": "cash",
  "vehicle_type": "bajaj",
  "route_km": 3.2
}
```

Success response:

```
{
  "success": true,
  "data": {
    "request_id": 88213,
    "fare": 150.00,
    "distance_km": 3.20,
    "surge": false
  },
  "v": "3.7.4"
}
```

(`request_id` is the new `rides.id`. An optional `quote_token` from a prior `ride_estimate_fare` call can be sent instead of raw lat/lng+route_km to price-lock the fare exactly as quoted.)

9.3 Login — `login`

(`AuthController.php`, no auth required)

Request:

```
{
  "action": "login",
```

```
  "phone": "0911000000",
  "password": "TestPassword123"
}
```

Success response:

```
{
  "success": true,
  "data": {
    "token": "a1b2c3d4e5f6...<opaque bearer token>",
    "user": {
      "id": 501,
      "name": "Test User",
      "phone": "0911000000",
      "role": "customer",
      "staff_role": null
    }
  },
  "v": "3.7.4"
}
```

(`token` is used as `Authorization: Bearer <token>` on all subsequent authenticated calls. `staff_role` is non-null only for EtelGroup-managed staff accounts.)

9.4 Wallet charge — `driver_topup_etel_money`

(`RideController.php` — the actual "charge the Etel Money wallet via Etel Pay" flow in this codebase; requires `Authorization: Bearer <token>`, any authenticated role. The driver first pays through the separate Etel Pay app/USSD, then submits the resulting transaction reference here for server-side verification.)

Request:

```
{
  "action": "driver_topup_etel_money",
  "amount": 150.00,
  "tx_ref": "ETP-TEST-000123456",
  "receipt_image_url": "https://ethelmarket.com/delivery/uploads/receipts/test-receipt.jpg"
}
```

Success response:

```
{
  "success": true,
  "data": {
    "tx_ref": "ETP-TEST-000123456",
    "amount": 150.00,
    "status": "pending",
    "receipt_image_url": "https://ethelmarket.com/delivery/uploads/receipts/test-receipt.jpg",
    "message": "Payment verified. Your top-up of ETB 150.00 is pending admin approval before it applies to your balance."
  },
  "v": "3.7.4"
}
```

(The transaction is verified against Etel Pay server-side via `etel_pay_verify_tx()` before this response is returned — a fabricated `tx_ref` fails with HTTP 402 and never reaches this shape. The balance only moves once an admin calls `admin_approve_etel_topup` — a deliberate two-step gate documented in the controller.)

Summary of code changes made in this pass

File	Change
`delivery/api/controllers/OrderController.php`	Added ownership checks to `check_pickup_confirmation` / `check_delivery_confirmation`

File	Change
	(IDOR fix); error_log added to two silent catches
`delivery/api/boot.php`	error_log added to 5 silent catches; normalized 8 pre-existing non-ASCII `·` separators to `-`
`delivery/api/controllers/AuthController.php`	error_log added to 2 silent catches
`delivery/api/lib/delivery_order_core.php`	error_log added to 2 silent catches (auto-dispatch)
`delivery/api/updater-api.php`	error_log added to audit-log-write catch
`delivery/api/backups/.htaccess`	**New** — deny-all + indexes-off lockdown
`delivery/js/core.js`	Added global `esc()` alias for `escapeHtml()` (fixes a real `ReferenceError` bug in `admin.js`, incidentally found while doing the XSS pass)
`delivery/js/admin.js`	Escaped 4 admin tables (Users/Service-Givers/Items/Couriers)
`delivery/js/customer.js`	Escaped storefront item grid/detail modal, messages, order list
`delivery/js/courier.js`	Escaped available/accepted job cards
`delivery/js/servicegiver.js`	Escaped incoming-orders list
`delivery/js/driver.js`, `delivery/js/ride.js`	Escaped ride-history detail modals
`delivery/js/*-bundle.js` and `delivery/js/releases/{driver/v6,ride/v5,customer/v1,courier/v1}/*.js`	Rebuilt via `node js/build-bundles.js` so the above fixes reach the live-served pages, not just source files
`.gitignore`	Added `node_modules/` (installed locally to run the terser-based bundl rebuild, not committed)

All PHP edits verified with `php -l` (no syntax errors) and the ASCII-safety check (no new non-ASCII bytes introduced). All JS edits verified with `node --check` (no syntax errors). Nothing was deployed — all changes are in this repo's git history only; deployment to `ethelmarket.com/delivery/` is a separate manual step through the Admin Updater tool.

Consolidated list of items needing Dagi's judgment call

Follow-up pass (post-submission-prep): Dagi made the call on all five items below. Status of each:

1. **RideController.php driver-privilege gap** (Section 1) — **FIXED.** Dagi's call: add the gate. A new `rides_require_onboarded($pdo, $uid)` helper (fails 403 unless `ride_drivers.onboarded = 1`) now guards every driver-money-moving/ride-accepting action: `driver_go_online`, `driver_get_nearby_requests`, `driver_accept_ride`, `driver_hail_customer`, `driver_start_ride`, `driver_arrived`, `driver_complete_ride`, `driver_cashout`, `driver_topup_etel_money`. A plain `customer`-role account (or a driver mid-onboarding) can no longer accept or run a real ride or move driver-side money. `driver_update_profile` (the onboarding submission endpoint itself) is deliberately NOT gated.
2. **`audit_agent.php`'s embedded AI-grading prompt** (Section 4) — **Left as-is.** Dagi's call: it's internal-only, never deployed, not part of the audit submission.
3. **`change_password` token-refetch failure** (Section 5) — **FIXED.** Dagi's call: hard-fail instead of soft-degrade. If the post-change `token_version` re-read fails, the endpoint now returns `{success:true, token:null, require_relogin:true}` instead of a token built from a guessed version. Both frontends (`admin.js`, `customer.js`) now check `require_relogin` and force an explicit re-login via `ETEL.auth.logout()` rather than silently showing "Password updated" with a token that might not work.
4. **Order code / zone-name escaping, and image/photo URL attribute escaping** (Section 3) — **FIXED.** Dagi's call: add it now as cheap insurance. Added a new `escUrl()` helper in `core.js` (HTML-attribute-escapes a URL and rejects `javascript:` / `vbscript:` / `data:text/html` schemes, allowing only `http(s):`, `data:image/*`, and relative paths) and applied it everywhere an image/photo URL was placed raw into a `src="..."` / `href="..."` attribute (`admin.js`, `servicegiver.js`, `customer.js`, `courier.js`, `driver.js`, `ride.js`). Also escaped every remaining unescaped `order_code` / `from_zone` / `to_zone` / `courier_name` / `customer_name` display spot found across `admin.js` (order table, dashboard table, pending-confirmations panel, confirmation history, Telebirr/bank payment lists) and `servicegiver.js` / `courier.js`, and hardened the remaining raw `order_code` JS-string-in-onclick builders with the same `.replace(/[<>"'\\]/g, '')` stripping pattern used elsewhere in this codebase.
5. **`js/releases/driver/v6/` appears to be missing several expected files** (Section 3) — **FIXED.** Confirmed real: `driver-bundle-extra.js`, `driver-rate-rider.js`, `driver-announcements-v2.js`, and `sos-button.js` were missing from both `js/releases/driver/v5/` and `v6/` (present in v1-v4, and byte-identical across all of v1-v4 — they were simply never copied forward when v5/v6 were cut). If the driver app's stable/canary channel is ever pointed at v5 or v6, any driver on that version would 404 on these four files and lose rating, in-app announcements, the SOS button, and premium/nav features. Restored by copying the v4 copies (identical content) into both v5 and v6.

All shipped JS (`core.js` , `admin.js` , `customer.js` , `courier.js` , `driver.js` , `ride.js` , `servicegiver.js`) was rebuilt via `node js/build-bundles.js` and synced into the currently-live release folders (`driver/v6` , `ride/v5` , `customer/v1` , `courier/v1`) the same way as the original XSS fix pass. Verified with `php -l` , `node --check` , and an ASCII-byte check on the touched PHP files.

---

10. Update — 2026-09-20: NFC card integration & other new surface since the pass above

Re-scoped to just the code that is genuinely new since the pass above, found by diffing every `case '<action>':` name currently in `delivery/api/boot.php` and every file under `delivery/api/controllers/` against every action name already inventoried in Section 8 of this document — 27 action names were new; 20 of those turned out to already be documented under a combined table cell the diff script split incorrectly (e.g. `` `get_items`, `customer_get_items` `` on one row), leaving **7 genuinely new actions** in one file (`NfcController.php`) plus **7 more** in a driver-compliance-documents feature added to `RideController.php` , plus **7 more** in a brand-new admin document library (`DocumentsController.php` + two standalone multipart endpoints). None of these three features existed, or were reviewed, when Sections 1-9 above were written.

10.1 `NfcController.php` (new file — `nfc_driver_status` , `nfc_start_ride` ,

`nfc_list_courier_orders` , `nfc_confirm_pickup` , `nfc_confirm_delivery`)

Checked: the full file (531 lines) plus its `boot.php` routing (the 5 action names are dispatched straight to this controller with no `require_auth()` first — correct, since these are server-to-server calls from EtelID after an NFC card tap, not calls from a Delivery-app bearer token, mirroring `AgentController.php` 's existing service-key pattern already verified clean in Section 1).

Findings: none. No fix required.

Verified clean: - Every case calls `nfc_require_service_key()` as its first line — a constant-time `hash_equals()` check against `ETELID_SERVICE_KEY` (header, with a `?key=` query-param fallback for hosts that strip custom `X-*` headers — same dual-path pattern already used elsewhere in this codebase per the shared skill notes). This is the sole trust boundary: nothing in this file is reachable without the shared key. - `nfc_start_ride` verifies the resolved driver's `role` is `driver` / `admin` **and** `ride_drivers.onboarded = 1` before letting them start a trip — the same onboarding gate Section 1's driver-privilege fix added to the phone-entry ride flow, applied here too so the NFC path can't be used to route around it. - `nfc_list_courier_orders` / `nfc_confirm_pickup` scope a service-giver confirmer to their own business via the same three-tier ownership check (direct `service_giver_id` link → `order_assignments` link → zone fallback) `sg_confirm_pickup` already uses and was verified clean for in Section 1 — no separate, weaker copy of this logic was introduced. - `nfc_confirm_delivery` requires an exact `orders.customer_id = $confirmer_user['id']` match — same ownership rule as `customer_confirm_delivery` . - `nfc_driver_status` returns driver compliance/license data (Section 10.2's data) with **no role check of its own** — this is intentional and matches the file's own header comment: EtelID already verifies the scanning person holds the "Traffic Officer" role (via its own Employees-panel data) before ever calling in here, and the service key is what proves the call genuinely came from EtelID having done that check. Documented in the file as a deliberate trust delegation, not an oversight — same shape as `AgentController` trusting the service key for `agent_*` actions in Section 1. - All SQL is parameterized; the one dynamically-built column list (`nfc_start_ride` 's optional `ride_code` / `request_code` / `code` / `tracking_code` / `order_code` / `vehicle_type` columns) is built only from a hardcoded whitelist array checked against `SHOW COLUMNS` , never from request input. - `nfc_find_or_create_user()` 's auto-provisioned account gets a `password_hash()` of random bytes (never a guessable/derivable value) and is only ever reachable through the same service-key boundary — not a path a public visitor can trigger directly.

10.2 Driver compliance documents (new actions in `RideController.php` :

`admin_list_driver_compliance` , `admin_update_driver_compliance` , `driver_get_compliance_status` , `driver_submit_compliance_document` , `admin_get_pending_compliance_requests` , `admin_approve_compliance_document` , `admin_reject_compliance_document`)

Checked: all 7 actions (lines ~1882-2084) plus the `driver_compliance_requests` table definition.

Findings: none. No fix required.

Verified clean: - Every `admin_*` action in this group requires `$u['role'] === 'admin'` **and** the `users.manage` (or `users.view` for the read-only list) capability — matches the established `require_admin($pdo, 'capability')` pattern everywhere else in this file. - The two driver-facing actions (`driver_get_compliance_status` , `driver_submit_compliance_document`) only ever read/write rows scoped to `$u['id']` (the authenticated caller from the file's module-level `require_auth()`) — a driver can only see or submit their own documents, never another driver's. - `admin_approve_compliance_document` builds `$expiryCol` / `$receiptCol` by concatenating `$req['doc_type']` into a raw `UPDATE ... SET $expiryCol=?, $receiptCol=?` string — looks like a dynamic-column-name pattern worth double-checking, but `doc_type` is doubly constrained: (1) the only INSERT path (`driver_submit_compliance_document`) whitelists it to exactly `insurance` / `technical_inspection` / `road_fund` via `in_array(..., true)` before the row is ever created, and (2) the column itself is a MySQL `ENUM('insurance','technical_inspection','road_fund')` (line 356), so even a hypothetical future INSERT path bypassing the PHP check would still be rejected at the database level. No fix needed — safe by construction, not by convention alone. - `expiry_date` / `requested_expiry_date` values are validated with `preg_match('/^\d{4}-\d{2}-\d{2}$/', $v)` before ever reaching a query.

10.3 Admin document library (new file `DocumentsController.php` — `admin_docs_list`, `admin_docs_get`, `admin_docs_create_text`, `admin_docs_update_text`, `admin_docs_update_meta`, `admin_docs_restore_version`, `admin_docs_delete` — plus two new standalone endpoints, `documents_upload.php` and `documents_download.php`, and a shared helper, `lib/documents_storage.php`)

Checked: all three files in full (`DocumentsController.php` 230 lines, `documents_upload.php` 260 lines, `documents_download.php` 201 lines, `documents_storage.php` 112 lines).

Fixed (1 finding — Medium/High severity: admin-to-admin stored XSS → session-token theft)

`documents_download.php` served `.html` / `.htm` / `.js` documents with `Content-Disposition: inline` by default, and the front-end viewer (`admin-documents.js` line 277) loads that response straight into an `<iframe src="...">` via a `blob:` object URL. A `blob:` URL created by a page inherits that page's own security origin — so an uploaded HTML document containing `<script>` would execute with the full origin privileges of `ethelmarket.com`, including read access to `localStorage.getItem('etel_token')` (`core.js` line 201 — the bearer token that *is* this app's entire session/auth mechanism, per Section 4's own findings on token-based auth).

Why this matters beyond "admin-only, so low risk": `documents.manage` (upload/replace/ edit) and `documents.view` (view/download) are two separate capabilities in the EtelGroup-managed role-permissions system — an account holding only `documents.manage` is not necessarily a full owner-tier admin. Under the previous behavior, such a scoped account could plant a malicious `.html` document, and **any higher-privileged admin who simply opened it to view it** (a completely ordinary action for a "review this document" workflow) would have their own real bearer token exfiltrated to wherever the script sends it — a privilege-escalation path from a narrow capability to full owner-tier account takeover, not merely a self-XSS.

Fix applied in `delivery/api/documents_download.php`: the actual extension of the file about to be streamed is now derived from `$storageName` (the server-generated `doc_<timestamp>_<hex>.<ext>` filename actually being read off disk — correct for both the current version and an older `version_id`, unlike `$doc['file_ext']`, which only reflects the document's *current* version and can disagree with an older one). If that extension is `html`, `htm`, or `js`, the response is forced to `Content-Disposition: attachment` regardless of the client-requested `disposition` query parameter — the viewer can never opt back into inline rendering for a type that can carry executable script. PDFs, Office docs, images, and genuinely-inert text types (md/txt/csv/json/yaml/xml/css) are unaffected and still render inline as before.

```
$streamedExt = strtolower(pathinfo($storageName, PATHINFO_EXTENSION));
if (in_array($streamedExt, array('html', 'htm', 'js'), true)) {
    $disposition = 'attachment';
}
```

Verified with `php -l` (no syntax errors).

Verified clean (the rest of the document library)

- **Path traversal / arbitrary file write:** storage filenames are always server-generated (`etel_docs_new_storage_name()` — `doc_<time>_<hex>.<ext>`), never derived from a client filename; every read (`etel_docs_read_text()`, `documents_download.php`) additionally wraps the stored name in `basename()` before building the filesystem path.
- **Upload type restriction:** `documents_upload.php` whitelists extensions against `etel_docs_type_map()` and additionally content-sniffs (`mime_content_type()`) to reject anything reporting a `php` / `x-httpd` / `x-executable` / `x-sh` / `x-python` / `x-perl` / `x-msdownload`-flavored MIME regardless of its extension — a disguised `report.md.php`-style double extension is still caught by `pathinfo()` only ever reading the last extension, and a renamed executable is still caught by the content sniff.
- **Auth on both standalone multipart/streaming endpoints:** both independently re-verify the bearer token (HMAC signature + 7-day expiry + live `token_version` match — same scheme Section 4 verified for the rest of the app), require `role === 'admin'`, and additionally check the `documents.manage` / `documents.view` capability for EtelGroup-managed accounts — not a weaker, separately-invented auth path just because these two files can't `require` `boot.php`'s controllers directly.
- **SQL:** fully parameterized throughout; `admin_docs_update_meta`'s dynamic `SET` clause is built only from a hardcoded whitelist of three column names (`title` / `category` / `description`), never from request keys.
- **Size caps:** 3 MB for inline text create/edit (`DocumentsController.php`), 20 MB for binary upload/replace (`documents_upload.php`).
- **Soft-delete only:** `admin_docs_delete` sets `deleted_at`, never actually removes the file from disk — consistent with this being an internal reference-document library where accidental permanent loss would be a real problem.

10.4 Other new-since-last-pass items checked, no fix needed

- `get_vapid_public_key` (`PushController.php`) — returns the app's public VAPID key for Web Push subscription. Public by design (a VAPID *public* key is meant to be client-visible; it has no authority without the paired private key, which never leaves the server) — correctly requires no auth.

All edited/new files in this section verified with `php -l`. No non-ASCII bytes were introduced (Delivery & Ride is not under the hard ASCII constraint that applies to EtelID/ EtelPay, but the check was still run for consistency).

Summary of code changes in this update

File	Change
`delivery/api/documents_download.php`	Force `Content-Disposition: attachment` for `.html` / `.htm` / `.js` documents regardless of requested disposition (stored-XSS-to-token-theft fix)

Needs Dagi's judgment call

None new. `NfcController.php`'s `nfc_driver_status` trusting EtelID's own role check with no independent re-verification (10.1) is a deliberate, already-documented architectural choice consistent with every other service-key bridge in this codebase, not a new item requiring a decision.